

sieve of eratosthenes

El algoritmo de la Criba de Eratóstenes es una herramienta fundamental para hallar todos los números primos en un segmento de 1 a N . Su lógica base consiste en asumir inicialmente que todos los números son primos y, al iterar desde el 2 en adelante, ir marcando los múltiplos de cada primo encontrado como compuestos, exceptuando siempre el 0 y el 1.

1. sieve

Para llegar a rangos de 10^7 , nuestro verdadero enemigo es la memoria. Para mitigar esto, aplicamos las siguientes optimizaciones matemáticas:

- Es suficiente iterar usando los números primos que no excedan $\sqrt{N}$.
- Al procesar un primo i , empezamos a tachar sus múltiplos desde i^2 .
- Dado que el único número primo par es el 2, todos los demás pares son compuestos. Al ignorar los pares, el tamaño del arreglo de memoria se reduce a la mitad. Para implementarlo, hacemos que el índice i represente al número impar $2i + 1$.

```
1  const int NN = 1e7;
2  vector<int> vp;
3  vector<bool> vv(NN / 2 + 1, true);
4  void sieve(){
5      vp.push_back(2);
6      for(long long i = 1; (2 * i + 1) * (2 * i + 1) <= NN; i++){
7          if(vv[i]){
8              long long p = 2 * i + 1;
9              for(long long j = 2 * i * (i + 1); j <= NN / 2; j += p)
10                 vv[j] = false;
11          }
12      }
13      for(int i = 1; i <= NN / 2; i++)
14          if(vv[i]) vp.push_back(2 * i + 1);
15 }
```

2. Bitwise Sieve

El problema principal de la criba estándar es el desperdicio de espacio a nivel estructural. En la mayoría de las arquitecturas, el tipo de dato booleano (`bool`) utiliza 1 byte completo de memoria. Sin embargo, para representar si un número es primo (1) o compuesto (0), en realidad solo necesitamos un único bit.

En lugar de crear un arreglo de booleanos, se crea un arreglo de números enteros. Dado que un número entero estándar tiene 32 bits, se pueden usar los bits individuales de ese entero para comprimir la información de 32 números distintos en un solo espacio

de memoria. Si sumamos esta optimización de bits con la exclusión de números pares, el tamaño del arreglo se reduce drásticamente a $N/64$. Esto permite empujar el límite hasta aproximadamente 10^8 .

```

1  const int N=1e8;
2  int p[N/64+1];
3  vector<int>pr;
4  #define check(n) (p[n>>5] & (1<<(n&31)))
5  #define set(n) (p[n>>5] |= (1<<(n&31)))
6  void bsieve(){
7      pr.push_back(2);
8      for(11 i=3; i*i<=N; i+=2) {
9          if(!check(i/2)) {
10             for(11 j=i*i; j<=N; j+=2*i){
11                 setBit(j/2);
12             }
13         }
14     }
15     for(int i=3; i<=N; i+=2){
16         if(!check(i/2)) pr.push_back(i);
17     }
18 }

```

3. Segmented Sieve

La Criba Segmentada se usa cuando necesitas hallar primos en un rango específico $[L, R]$. El límite superior R puede ser inmensamente grande, soportando valores de hasta 10^{12} .

Sin embargo, el tamaño de tu "ventana" de memoria siempre será exactamente $(R - L + 1)$. Generalmente, las restricciones de los problemas indicarán que la diferencia entre L y R no supere los 10^5 o 10^6 .

La lógica consiste en precalcular los primos hasta $\sqrt{R}$ con una criba normal, y usar esos primos para tachar los múltiplos únicamente dentro del rango $[L, R]$. Como nuestro arreglo local empieza en el índice 0, mapeamos el valor j al índice $j - L$.

```

1  void segsieve(11 L, 11 R){
2      vector<bool>vv(R-L+1, true);
3      if(L==1) vv[0]=false;
4      for(int p:vp){
5          if(1LL*p*p>R) break;
6          11 st=max(1LL*p*p, (L+p-1)/p*p);
7          for(11 j=st; j<=R; j+=p){
8              vv[j-L]=false;
9          }
10     }
11     for(11 i=0; i<=R-L; i++){
12         if(vv[i]){
13             cout<<(i+L)<<'\\n';
14         }
15     }
16 }

```